Efficient Multi-Scale Indexing with Dynamic IntMaps

Albert Eisfeld[1], Stephan Schulz[1]

[1]DHBW Stuttgart, Stuttgart, Germany

Abstract

Term and clause indexing are central technologies for efficient first-order theorem provers and similar reasoning systems. While there are a selection of basic technologies, most of them require an efficient mapping from function symbols (usually encoded as small integers) or other integer values to branches of a tree structure. With today's wide variety of application problems, both the range of possible values and the cardinality of each individual map vary wildly. We present a data structure that is designed to be efficient in time and space at all scales and key distributions. Experiments show very good scalability and a modest improvement over the old, already quite refined data structure in E.

Keywords

term indexing, data structures, theorem proving,

1. Introduction

Most powerful modern theorem provers like E [1], Vampire [2] and iProver [3] are based on saturating calculi, in particularly variations of superposition [4] and related calculi. These calculi represent the proof state as a set of clauses. They enrich the proof state by adding logical consequences of existing clauses, with the aim of eventually generating the empty clause as an explicit witness of unsatisfiability (and hence conclude a proof by contradiction). In addition to generating inferences, simplifying inferences like subsumption and rewriting are critical for success. These techniques either remove redundant clause, or replace them by simpler ones, and typically take up most of the time spent by the inference engine.

Successful provers combine efficient execution and intelligent proof search heuristics to achieve their power. Especially for harder problems with large search states, *term indexing* is a critical technology. Indexing of terms (and of clauses) allows a system to quickly identify potential inference partners for generating inferences like paramodulation/superposition or resolution, but in particular also for simplification techniques like rewriting and subsumption.

Most modern term indices are based on (often trie-like) trees. At each node in the tree, branches labelled with function symbols (or variables) lead to sub-trees. A complete branch of the tree represents a query compatible with a certain query, and the leaves represent sets of terms (or clauses) that are potential answers to this query. In this paper, we are in particular concerned with the efficiency of mapping function (or variable) symbols to tree branches. However, the developed solutions have more general applications, both in indexing and beyond.

2. Background and Motivation

Early contributions to indexing were made by Stickel [5], who introduced *path indexing*, and by McCune, who integrated path indexing and *discrimination tree indexing* into Otter [6], creating the first successful modern saturating prover. Our own prover E uses *perfect discrimination tree indexing* for forward rewriting, *fingerprint indexing* [7] for backwards rewriting and superposition/paramodulation, and *feature vector indexing* [8] for subsumption and related techniques.

10th Workshop on Practical Aspects of Automated Reasoning, Lisboa, 2026

inf23040@lehre.dhbw-stuttgart.de (A. Eisfeld); schulz@eprover.org (S. Schulz)

0000-0001-6262-8555 (S. Schulz)

© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

When indexing techniques were first developed, most problems for automated theorem provers were handcrafted, and small in number of axioms and signature size. Indexing data structures, mapping (mostly) from function symbols to branches of the index tree, hence usually had a low branching factor and also a very limited range of values to branch on. Implementations of these maps used either (potentially dynamic) arrays with direct mapping, or linear lists with linear search. Arrays are faster, but if the node is not densely populated, may waste space. Linear lists have a higher per-key cost, both in speed and memory, but handle sparse key spaces efficiently.

In E, our original implementation used two dynamic arrays to handle each node - one for variable-indexed branches, one for function-symbol indexed branches. This worked well for most problems of the time. However, in 2003, Urban was converting the MIZAR library [9] to first-order logic [10, 11] and began experiments running E on the resulting problems. MIZAR problems have many thousands of function symbols, and the resulting large, but sparsely populated arrays led to an explosion of memory consumption by E's perfect discrimination tree indices (at that time the only indexing data structure used). Urban solved this by hacking the prover and replacing the function symbol array with a binary search tree. While this solved his immediate problem, it decreased performance for other classes of problems, and was the optimal solution for all situations.

However, inspired by Urban's work, we replaced both the original solution and Urban's hack with a dedicated `IntMap` data type. It's abstract interface supports inserting and deleting key/value pairs, querying the value for a given key, and iteration over all or an arbitrary sub-range of key/value pairs stored in a given map.

Before we discuss the details of the original and the new `IntMap`, we present a couple of observations that shape the design.

First, the proof state of saturating theorem provers tends to increase in size over time, and so does the number of indexed clauses. There are occasional cases where already indexed clauses become redundant and have to be removed from the index (*backward simplification*), but this is a rare occurrence. Typically, all indices (and all `IntMaps` used in them) tend to grow over time.

The most frequent operation by far is the query for a given key (and its associated value). Insertions are orders of magnitude more frequent than deletions. In-order iteration over key/value pairs is a frequent operation as well - more frequent than insertions, but less frequent than look-ups.

Our normal use case are term and clause indices. These are typically arranged as trees, with each node of the tree representing a set of terms or clauses, and each branching splitting this set into disjoint subsets. Each branch is typically labelled by a small integer (the *key*), often representing the encoding of a function symbol (i.e. its index in the signature table) or a variable (typically a small negative number), and resulting from test on the terms in that branch. As a result, the sets (and the branching factors) of the early nodes are often quite large, but as we follow the branches, the subsets and hence the branching factors get smaller and smaller. Earlier nodes are accessed much more often (dominating access times), but there are many more nodes deeper in the tree, which are thus contributing significantly to memory use.

While the branching factor goes down with depth, the range (i.e. the difference between the smallest and the largest key) only goes down statistically, and not to the same degree. We have fewer keys, but drawn from the same total range. This range varies a lot between problems. Variables are typically restricted to the low dozens. Function symbols (and predicate symbols) can vary from single operators to tens of thousands for problems from Mizar and SUMO [12], and over a million for example problems from Cyc [13].

These observations lead us to the conclusion that the mapping of keys (integers) to branches in the indices must be efficient in time and memory at many different scales.

3. The Evolution of IntMap

As stated above, our first implementation of branching in perfect discrimination trees relied on two separate dynamic arrays (a generic datatype already implemented in E's libraries), with direct mapping

from positive `long int` keys to `void*` values. These supported only 0-based indexing. Urban replaced the function symbol side of the index by `NumTrees`, another of E's built-in data types. `NumTrees` are one of several implementations of Splay trees [14] in E, in particular Splay trees that use `long int` keys and either `long int` or `void*` values. Splay trees are structurally normal binary search trees, and can be used as such. However, most operations that require key lookup rearrange the tree, to move the requested key or one of its neighbours to the root of the tree. As a result, Splay trees provide an amortised logarithmic cost guarantee, and in particular, if only a subset of n keys is frequently accessed, essentially behave like a tree of only these n keys.

We then introduced `IntMap` as an abstract data type. It supports insertion, deletion, lookup, and ordered iteration via an iterator data type with *allocate*, *next* and *free* operations.

This `IntMap` datatype had a polymorphic implementation, with 4 distinct cases:

- *Empty*: The initial state on creation. No key/value pair is stored. The iterator for an empty `IntMap` simply returns an unspecified key and value `NULL`, the marker for the end of iteration.
- *Single*: A frequent case in indexing are long branches created by a single indexed term. To accommodate this case efficiently, we provide an implementation that stores a single key and associated value. The iterator for this case uses a boolean value to indicate if this one key/value pair has been seen already. If not, it is returned, if so, it returns `NULL` (with an unspecified key) again.
- *Array*: Dense populations of multiple keys are stored in a `PDRangeArray`. This is a flexible dynamic array type from E's library. It allows indexing with `long int` values starting at an arbitrary (potentially negative) offset. This array automatically grows as needed, but never shrinks (remember that deletions are rare events). The iterator for the array case maintains the index of the next unseen array element. The *next* operation searches for the next non-`NULL` value in the array and returns the corresponding key/value pair, or `NULL` if runs out of array elements.
- *Tree*: Sparse key/value populations are stored in a `NumTree`, as described above. The iterator uses a stack to describe the current state of iteration, and uses a non-destructive in-order (Left-Node-Right) traversal to access all nodes. It returns the stored key/value pair at each node, or `NULL` if it runs out of nodes.

In both of the multi-key-modes, the `IntMap` monitors population density. If the density falls below a certain threshold value, and the map is in array mode, it reorganises itself into tree mode. If the density in tree mode increases beyond a second (larger) threshold, it switches from tree to array mode. The difference in the two thresholds provides a hysteresis that avoids overly frequent reorganisations. In our implementation, we switch to array if more than one in 8 spaces are occupied, and we switch to tree if less than one in 4 spaces are occupied. When first switching to multi-key-form with density in the hysteresis range, array is preferred (on the assumption that the density is more likely to increase than to fall).

3.1. Recent Improvements

Observations have shown that the key-value-pairs often lie in the memory as clusters, so that there are some smaller regions which are significantly denser populated than the overall set of all key-value-pairs considered for a tree. A possible reason for this is that in larger specifications, there are sub-theories mostly dealing with a subset of functions and relations, while also being tied into the common theory with some shared symbols. This is then reflected in indexing data structures for clauses from these sub-theories.

To accommodate this circumstance, the structure of the `IntMap` datatype was simplified to use only one case instead of the two *Array* and *Tree* for multi-key-modes, combining both of the older cases into one. Key/Value pairs are stored in a tree of smaller arrays. This new *Array-Tree* is based on the `NumXTree` datatype, a binary tree that contains arrays with a size of 2^n entries in each node. Keys are effectively split into two parts, one part (the high bits) selecting the array in the tree, the other (the low n bits) the index of the value in the array.

Figure 1 shows the visualisation of an exemplary set of key-value-pairs. The number next to the node represents the key of the tree node and thus the high bits of each key in that array. The index in the array represents the low 2 bits of the key.

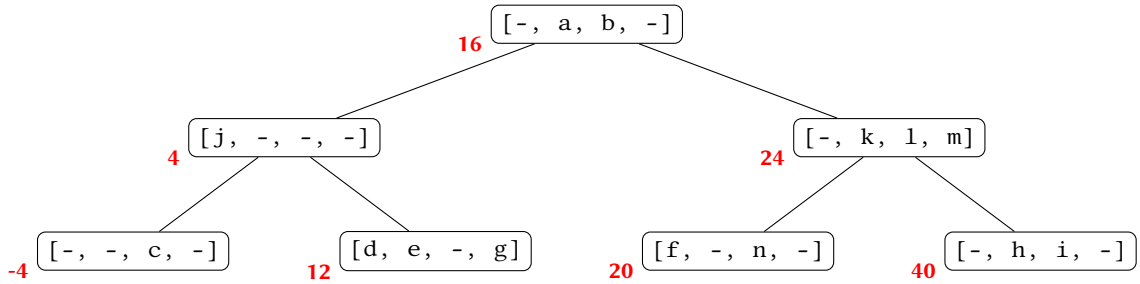

Figure 1: Visualisation of exemplary `NumXTree` binary tree with an array size of 4. Values in the array that are marked with '-' represent `NULL`-values, i.e. these keys are not associated with any value.

Using the `NumXTree` binary tree, the new `IntMap` datatype still supports insertion, deletion, lookup and ordered iteration as in the older version.

Similar to the *Tree* case, the ordered iteration uses a non-destructive in-order traversal. A stack is used to keep track of the current array in the tree. Because of the nested structure, iterating over the tree nodes alone would yield only values at index 0 of the array. To iterate over all key-value-pairs, the iterator uses an additional index counter to indicate the next unused entry in the current array. Iteration is hence done at two levels: First, the iterator looks for the next occupied entry in the current array. If it runs out of possibilities, the iterator moves to the next tree node and resets the internal index counter. In terms of memory, the iterator combines both the stack and the array index of the old model, thus using slightly more memory. However, iterators are rare objects, and the (dynamic) stack can typically be smaller, since there are fewer tree nodes.

Using the exemplary tree from Figure 1, an iterator would return the values in the following order: `↑c, ↑j, d, e, g, ↑a, b, ↑f, n, ↑k, l, m, ↑h, i`, where each of the ↑-occurances marks transition to a new tree node. The values have the keys depicted in Table 1.

Value	c	j	d	e	g	a	b	f	n	k	l	m	h	i
Key	-2	4	12	13	15	17	18	20	22	25	26	27	41	42

Table 1
Key-value-pairs of exemplary `NumXTree` binary tree in Figure 1

The proposed `NumXTree` binary tree comes with benefits for memory and runtime. Each node now can store multiple key-value-pairs while still needing only two words of memory for the left and right child-nodes to maintain the tree structure. Furthermore, the amount of operations for rearranging the tree is reduced on average, since fewer nodes can hold the same number of values. Additionally, reorganising the `IntMap` when the critical density is reached is now obsolete, since there is only one multi-key-mode.

Since the structure is optimised on the common case, it is expected to perform worse than the original structure in runtime and memory in edge cases for the key/value distribution. If the key-value-pairs are either very densely or very sparsely populated, organising the pairs in an array or a binary tree respectively will be optimal for memory and in the case of the array state, also in runtime. However, on average we expect better behaviour, and in particular better worst-case behaviour.

4. Experimental Results

To examine whether the changed data structure actually improves prover runtime and memory footprint, we ran the prover on various problems from the TPTP library [15], version 9.2.1. In order to provide a sufficient comparison, the theorem prover was built using the original state of the `IntMap` data structure

with two states for organising multiple key-value-pairs, as well as six versions of the improved `IntMap`, with array-sizes of 1, 2, 4, 8, 16 and 32.

All versions of the program were built using the gcc compiler with the optimisation flags `-O03 -fno-common`. The tests have been done on a Linux server with Intel Xeon Platinum 8176 CPUs running at 2.10GHz under Ubuntu 24.04.3 LTS. The machine had 1.5TB physical RAM. The source code is available at https://github.com/aallbert/eprover.

To obtain the time used to find the proof for the according problem, the Linux `time` command was used. All problems have been run on their own core with a soft runtime limit of 300 seconds to ensure consistency and controlled termination. The command flags used are the following: `time taskset -c 32-63 ./eprover –auto –detsort-new –detsort-rw –soft-cpu-limit=300 –cpu-limit=350 <problem>`

We picked 158 problems from the First-order Theorems category of the CADE ATP System Competition [16] (see Table 7) (years 2025 - 2019) to give us an interesting selection of problems with a good distribution of difficulties.

From these problems, we excluded all problems that finished below 5 seconds for all versions, to avoid measurement noise. Problem runs over 300s indicate that the prover did not find a proof in time and were also excluded from analysis.

This leaves 34 problems with valid data, where at least one version of the prover runs in between 5 and 300 seconds. The times for these problems are shown in Table 4. The number of times how often each version was the fastest in finding a proof and the total runtime for all 34 problems are depicted in Table 2.

Array Size	**Fastest Version**
4	11
2	10
Standard Version	9
16	2
1	1
8	1
32	0

Array Size	**Total Time spent (s)**
8	3158.944
16	3234.791
Standard Version	3314.828
1	3325.156
32	3366.841
4	3390.069
2	3591.788

Table 2
Number of times each version of the data structure was fastest (left) and total time spent on the 34 problems (right). Note that the *Standard Version* refers to the `IntMap` data structure that uses the binary tree and array to organise the key-value-pairs.

After looking at the runtime distribution of different functions in the program using a profiler, a maximum improvement of 2% was expected, since all functions related to the `IntMap` take up roughly 2% of the runtime for each problem. Table 2 shows that the differences in total runtime in the right table are unexpectedly larger than 2%. Additionally, in the set of 34 valid problems are some outliers where the time differences in between versions are greater than 80% as shown in Table 4. This effect has occurred before, and indicates that the proof search (which can depend on in-memory allocation of pointers) took a different part. These outliers hence are not suitable to evaluate the quality of the data structures.

The results show that the versions with an array size of 1 and 32 perform worse than the Standard Version in both metrics. Surprisingly, the number of times a version solved a problem the fastest does not correlate to its overall runtime. As demonstrated in Table 2, the versions with an array size of 2 and 4 score high in the number of times they are the fastest, but low in the total time they spent on the problems. Note that versions 2 and 4 have more negative and positive outliers, i.e. they reach the 300 seconds run time limit where other versions complete the problem in under 120 seconds and vice versa. Table 5 shows that when excluding positive and negative outliers, versions 2 and 4 perform much better in overall runtime compared to the Standard Version. An array size of 8 provides a stable version that performs best in runtime with all valid problems included and outperforms the Standard Version in

runtime even when excluding the outliers.

For the 34 valid problems, the memory consumption, i.e. how much memory was allocated to find a proof, was also monitored. The results are shown in Table 3.

Array Size	**Least Memory**
Standard Version	20
8	9
32	4
16	1
4	0
2	0
1	0

Array Size	**Total Memory allocated (GB)**
Standard Version	4198.438
8	4360.340
32	4415.674
16	4468.586
4	4470.512
2	4561.600
1	4585.171

Table 3
Number of times each version of the data structure consumed the least amount of Memory (left) and total memory allocated on the 34 valid problems (right). Note that the *Standard Version* refers to the `IntMap` data structure that uses the binary tree and array to organise the key-value-pairs.

None of the improved versions show a fewer overall memory consumption compared to the standard version of `IntMap`. This suggests that the time savings do not directly correlate with the amount of memory consumed and are more dependent on functions that require heavy computation.

5. Future Work

Our current implementation is already performing quite well. There are several options to further improve the data structure, but none that is likely to make a significant impact in overall prover performance. The most attractive idea, if primarily from a simplicity and aesthetics perspective, would be to use dynamically sized array segments in the current array tree structure, i.e. to merge neighbouring arrays if the resulting larger array would be full enough, and likewise to split them if the population factor sinks below a threshold. This would allow an even more smooth transition between all-array and all-tree representations.

Another idea would be to analyse the problems on typical patterns and set the array size according to these patterns. Since some problems show drastic outliers, changing the array size after the prover read all axioms could have a significant impact on the runtime of some cases.

6. Conclusion

In older work, Nieuwenhuis, Hillenbrand, Riazanov and Voronkov have found that, back in 2001, the exact choice of indexing mechanism for a given retrieval operation was less important than the decision to pick any over none [17], and that details of implementation are not usually critical. Urban's experience shows that this did not hold up for all cases if the landscape of problems addressed by automated theorem provers changes significantly, e.g. with the introduction of much larger real-world problems (which lead, among other things, to the idea of a-priory premise selection and the *Workshop on Empirically Successful Automated Reasoning in Large Theories* (ESARLT) [18], held with CADE-21 in 2007 in Bremen). However, in this work the idea has reasserted itself - the new approach shows some advantages, but, at least in our test cases, they seem fairly marginal in run time. With the improved `IntMap` data structure, a slight but significant gain in speed has been achieved when considering all valid test-problems, though causing an increased memory footprint. Besides its impact on runtime, the improved `IntMap` also delivers a more elegant data structure, where switching the organisation between a binary tree and an array becomes obsolete.

Declaration on Generative AI

The author(s) have not employed any Generative AI tools.

References

[1] S. Schulz, S. Cruanes, P. Vukmirović, Faster, higher, stronger: E 2.3, in: P. Fontaine (Ed.), Proc. of the 27th CADE, Natal, Brasil, number 11716 in LNAI, Springer, 2019, pp. 495–507.

[2] F. Bártek, A. Bhayat, R. Coutelier, M. Hajdú, M. Hetzenberger, P. Hozzová, L. Kovács, J. Rath, M. Rawson, G. Reger, M. Suda, J. Schoisswohl, A. Voronkov, The Vampire diary, in: R. Piskac, Z. Rakamaric (Eds.), Computer Aided Verification - 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part III, volume 15933 of *Lecture Notes in Computer Science*, Springer, 2025, pp. 57–71. doi:10.1007/978-3-031-98682-6_4.

[3] A. Duarte, K. Korovin, Implementing superposition in iProver (system description), in: N. Peltier, V. Sofronie-Stokkermans (Eds.), Proc. of the 10th IJCAR, Paris (Part II), volume 12167 of *LNAI*, Springer, 2020, pp. 158–166.

[4] L. Bachmair, H. Ganzinger, Rewrite-Based Equational Theorem Proving with Selection and Simplification, Journal of Logic and Computation 3 (1994) 217–247.

[5] M. E. Stickel, The Path-Indexing Method for Indexing Terms, Technical Note 473, Artificial Intelligence Center, SRI International, Menlo Park, California, USA, 1989.

[6] W. McCune, Experiments with Discrimination-Tree Indexing and Path Indexing for Term Retrieval, Journal of Automated Reasoning 9 (1992) 147–167.

[7] S. Schulz, Fingerprint Indexing for Paramodulation and Rewriting, in: B. Gramlich, U. Sattler, D. Miller (Eds.), Proc. of the 6th IJCAR, Manchester, volume 7364 of *LNAI*, Springer, 2012, pp. 477–483.

[8] S. Schulz, Simple and Efficient Clause Subsumption with Feature Vector Indexing, in: M. P. Bonacina, M. E. Stickel (Eds.), Automated Reasoning and Mathematics: Essays in Memory of William W. McCune, volume 7788 of *LNAI*, Springer, 2013, pp. 45–67.

[9] A. Trybulec, H. A. Blair, Computer assisted reasoning with MIZAR, in: Proc. of the 9th IJCAI, Los Angeles, volume 85, Morgan Kaufmann, 1985, pp. 26–28.

[10] J. Urban, Translating Mizar for first order theorem provers, in: A. Asperti, B. Buchberger, J. H. Davenport (Eds.), Mathematical Knowledge Management, Springer, 2003, pp. 203–215.

[11] J. Urban, MPTP - motivation, implementation, first experiments, Journal of Automated Reasoning 33 (2004) 319–339. URL: https://doi.org/10.1007/s10817-004-6245-1. doi:10.1007/s10817-004-6245-1.

[12] I. Niles, A. Pease, Toward a Standard Upper Ontology, in: C. Welty, B. Smith (Eds.), Proc. 2nd International Conference on Formal Ontology in Information Systems (FOIS-2001), 2001, pp. 2–9.

[13] D. Ramachandran, P. Reagan, K. Goolsbey, First-orderized ResearchCyc: Expressiveness and Efficiency in a Common Sense Knowledge Base, in: P. Shvaiko (Ed.), Proc. of the AAAI Workshop on Contexts and Ontologies: Theory, Practice and Applications (C&O-2005), 2005.

[14] D. Sleator, R. Tarjan, Self-Adjusting Binary Search Trees, Journal of the ACM 32 (1985) 652–686.

[15] G. Sutcliffe, The TPTP problem library and associated infrastructure - from CNF to TH0, TPTP v6.4.0, Journal of Automated Reasoning 59 (2017) 483–502. doi:10.1007/s10817-017-9407-7.

[16] G. Sutcliffe, The CADE ATP System Competition – CASC, AI Magazine 37 (2016) 99–101. doi:10.1609/aimag.v37i2.2620.

[17] R. Nieuwenhuis, T. Hillenbrand, A. Riazanov, A. Voronkov, On the Evaluation of Indexing Techniques for Theorem Proving, in: R. Goré, A. Leitsch, T. Nipkow (Eds.), Proc. of the 1st IJCAR, Siena, volume 2083 of *LNAI*, Springer, 2001, pp. 257–271.

[18] G. Sutcliffe, J. Urban, S. Schulz (Eds.), Proceedings of the CADE-21 Workshop on Empirically Successful Automated Reasoning in Large Theories, volume 257 of *CEUR Workshop Proceedings*, 2007.

7. Appendix

Problem	Std.	S1	S2	S4	S8	S16	S32
ITP015+1.p	0.076	0.112	307.674	0.069	0.137	0.193	0.166
ITP013+1.p	0.162	0.161	306.51	304.999	0.157	0.156	0.163
NUM461+2.p	11.443	11.519	0.678	0.071	7.098	9.299	9.627
SYO606+1.p	46.245	46.688	0.091	9.234	14.102	48.293	51.437
SWW478+7.p	5.439	5.272	5.314	5.44	5.689	5.576	6.103
SEU348+2.p	7.152	7.235	7.217	7.237	7.433	7.478	7.935
PRO003+2.p	8.063	8.129	8.135	7.832	8.217	8.032	8.659
GRP746+1.p	306.37	306.48	16.701	307.238	307.567	306.824	306.486
SWW348+1.p	18.94	19.22	19.68	18.772	19.257	19.454	20.487
SWW350+1.p	19.193	19.398	19.694	19.161	19.657	19.557	19.944
SWW337+1.p	20.907	20.797	20.613	20.449	20.679	20.118	21.34
SWW339+1.p	21.705	22.06	22.238	21.848	21.897	22.057	22.561
GEO283+1.p	22.931	23.052	25.116	25.043	26.072	23.628	24.997
GEO286+1.p	28.884	29.085	30.16	29.611	31.799	29.576	31.16
GRP655+2.p	33.715	34.033	31.437	33.891	34.223	33.109	34.283
GEO281+1.p	31.721	31.889	35.234	33.036	34.847	32.933	34.539
PUZ077+1.p	50.433	50.232	35.081	42.176	45.367	48.507	52.393
GEO168+1.p	44.031	44.452	44.123	43.539	44.053	43.94	46.782
SYO605+1.p	53.539	53.839	50.49	52.702	53.238	53.46	55.582
LCL403+2.p	57.142	58.323	58.932	57.116	58.734	57.759	59.571
SET671+3.p	113.988	112.306	304.77	83.846	111.828	111.754	116.296
LCL642+1.010.p	81.422	82.259	84.527	83.955	84.406	86.174	91.24
SWW470+1.p	227.801	224.679	89.866	238.68	226.527	222.825	238.192
GRP654+2.p	95.475	96.542	89.605	90.596	92.278	95.299	99.265
GRP664+1.p	113.49	112.5	105.474	107.302	102.901	110.015	114.988
MGT005+2.p	216.325	217.314	113.361	110.072	114.986	114.329	121.136
LCL642+1.015.p	122.743	123.628	126.232	126.597	126.383	130.866	137.397
SWW893+1.p	140.615	140.882	307.357	124.601	128.324	138.03	146.766
SWW884+1.p	151.058	151.878	143.756	141.206	142.347	149.41	155.895
LCL656+1.015.p	164.727	164.912	165.782	166.106	168.33	170.198	178.473
GEO310+1.p	222.817	224.626	216.924	235.653	242.102	229.157	239.72
SCT117+1.p	293.299	293.225	221.799	255.493	264.382	285.535	305.842
GEO311+1.p	298.235	302.223	273.717	283.145	293.984	304.098	303.921
GEO329+1.p	284.742	286.206	303.5	303.353	299.943	297.152	303.495

Table 4
Time spent of each version on the 34 valid problems, where at least one version is in between 5 and 300 seconds runtime. Problems, where the time differences in between the versions are greater than 80% are seen as outliers and marked blue. These outliers have been tested three times each and the average time of each triple is counted as the valid data point and presented in the table. *Std.* is the Standard Version of the `IntMap` data structure, *S1*, *S2*, etc. are the versions that use the `NumXTree` binary tree with the according array size

Array Size	Total Time spent (s)
2	2144.780
4	2211.259
8	2248.218
Standard Version	2251.803
1	2265.015
16	2283.088
32	2376.572

Table 5: Total time spent on the valid problems where positive and negative outliers are *excluded*.

Table 6: Amount of memory each version allocated in GB for solving each problem - *Std.* is the Standard Version of the `IntMap` data structure, *S1*, *S2*, etc. are the versions that use the `NumXTree` binary tree with the according array size

Problem	**Std.**	**S1**	**S2**	**S4**	**S8**	**S16**	**S32**
SYO606+1.p	35.333	55.047	0.176	8.478	12.748	56.155	55.652
ITP015+1.p	0.245	0.248	163.571	0.248	0.248	0.248	0.248
NUM461+2.p	25.964	26.978	2.14	0.368	16.985	22.245	22.246
ITP013+1.p	0.465	0.48	124.072	133.904	0.48	0.48	0.48
SWW478+7.p	2.28	2.368	2.368	2.372	2.376	2.378	2.379
SEU348+2.p	8.957	13.099	13.151	13.253	13.217	13.218	13.219
PRO003+2.p	9.326	12.146	12.048	12.088	12.155	12.137	12.244
GRP746+1.p	222.106	224.469	20.407	231.762	223.546	233.831	216.401
SWW350+1.p	25.556	31.467	32.221	31.758	31.771	31.798	31.802
SWW348+1.p	25.568	31.434	32.219	31.628	31.714	31.743	31.747
GEO283+1.p	27.234	30.57	35.235	35.614	36.254	30.864	31.059
SWW339+1.p	29.552	35.071	35.499	35.649	35.725	35.732	35.09
SWW337+1.p	29.574	35.243	34.614	34.555	34.579	34.444	33.834
GEO281+1.p	32.008	35.275	41.896	43.628	44.244	36.034	36.062
GRP655+2.p	34.512	34.617	32.734	35.525	35.525	34.617	34.617
GEO286+1.p	38.316	41.88	44.961	45.593	47.098	42.567	42.568
SYO605+1.p	39.361	41.763	40.299	41.718	41.93	41.976	41.979
LCL642+1.010.p	47.259	79.731	81.029	80.941	81.072	81.203	81.204
LCL642+1.015.p	79.313	120.275	121.37	122.036	122.311	122.598	122.6
GEO168+1.p	83.845	94.012	93.728	94.448	94.468	94.517	94.517
GRP654+2.p	92.35	92.608	86.487	88.018	88.018	92.608	92.608
PUZ077+1.p	101.016	121.403	86.5	102.297	109.59	114.794	118.731
GRP664+1.p	107.029	107.37	101.677	104.205	99.04	107.37	107.367
SET671+3.p	113.605	121.085	305.847	99.883	121.254	121.256	121.212
LCL403+2.p	119.176	121.001	121.023	121.022	121.013	121.038	121.016
MGT005+2.p	238.389	275.035	144.914	145.39	147.425	147.502	147.505
LCL656+1.015.p	149.305	191.7	192.461	193.714	194.399	194.517	194.52
SWW893+1.p	199.778	202.139	563.753	171.233	171.254	202.618	202.622
SWW470+1.p	436.471	438.26	185.103	462.062	437.204	438.32	438.321
SCT117+1.p	329.992	358.768	290.908	338.988	337.579	356.363	355.335
SWW884+1.p	308.349	309.419	292.029	292.007	292.136	309.593	309.596
GEO310+1.p	361.234	389.262	371.507	410.379	411.069	394.101	394.083
GEO329+1.p	396.097	436.131	407.206	424.934	436.106	435.894	417.031
GEO311+1.p	448.873	474.817	448.447	480.814	475.807	473.827	455.779

Table 7: Time spent of each version on each problem - *Std.* is the Standard Version of the `IntMap` data structure, *S1*, *S2*, etc. are the versions that use the `NumXTree` binary tree with the according array size.

Problem	**Std.**	**S1**	**S2**	**S4**	**S8**	**S16**	**S32**
SWB053+1.p	0.013	0.012	0.011	0.01	0.009	0.008	0.007
NUN066+1.p	0.01	0.009	0.008	0.008	0.009	0.009	0.007
COM135+1.p	0.011	0.01	0.009	0.009	0.009	0.009	0.007
KRS261+1.p	0.011	0.01	0.009	0.007	0.008	0.008	0.009
KRS260+1.p	0.012	0.011	0.009	0.009	0.008	0.009	0.009

NUN075+2.p	0.013	0.013	0.06	0.094	0.009	0.008	0.01
NLP261+1.p	0.011	0.011	0.009	0.009	0.009	0.009	0.008
SWB046+1.p	0.011	0.01	0.009	0.009	0.009	0.008	0.009
LCL501+1.p	0.011	0.01	0.009	0.009	0.009	0.009	0.008
COM143+1.p	0.012	0.011	0.01	0.009	0.008	0.009	0.01
ITP015+2.p	0.01	0.012	0.009	0.009	0.01	0.009	0.008
REL044+1.p	0.011	0.012	0.009	0.009	0.008	0.01	0.009
SWV448+1.p	0.013	0.012	0.011	0.01	0.01	0.058	0.008
AGT009+2.p	0.011	0.01	0.009	0.009	0.008	0.01	0.009
GEO450+1.p	0.012	0.011	0.01	0.01	0.01	0.009	0.008
SWB025+1.p	0.012	0.009	0.011	0.009	0.009	0.009	0.008
AGT013+2.p	0.011	0.01	0.008	0.008	0.009	0.01	0.009
SWW096+1.p	0.012	0.011	0.009	0.008	0.01	0.008	0.01
SWB010+1.p	0.01	0.01	0.009	0.01	0.009	0.01	0.008
LCL558+1.p	0.011	0.009	0.009	0.009	0.008	0.009	0.009
KLE111+1.p	0.011	0.01	0.009	0.009	0.008	0.008	0.008
COM131+1.p	0.011	0.01	0.009	0.008	0.009	0.009	0.01
LAT376+3.p	0.011	0.008	0.009	0.01	0.01	0.009	0.008
ITP006+5.p	0.011	0.01	0.009	0.009	0.01	0.009	0.009
KLE170+1.004.p	0.011	0.01	0.009	0.009	0.009	0.009	0.01
SWC002+1.p	0.059	0.013	0.093	0.011	0.129	0.009	0.165
GEO474+1.p	0.07	0.109	0.146	0.19	0.009	0.231	0.01
CSR029+4.p	0.013	0.012	0.011	0.009	0.01	0.01	0.009
CSR038+2.p	0.012	0.011	0.01	0.009	0.01	0.009	0.009
NUN085+1.p	0.013	0.012	0.011	0.01	0.009	0.01	0.009
AGT023+2.p	0.011	0.01	0.009	0.009	0.01	0.009	0.009
REL033+4.p	0.012	0.011	0.009	0.009	0.009	0.01	0.009
AGT012+2.p	0.011	0.011	0.009	0.009	0.009	0.009	0.009
AGT022+1.p	0.011	0.01	0.009	0.009	0.01	0.009	0.009
ITP019+5.p	0.011	0.01	0.009	0.009	0.009	0.01	0.009
ITP020+2.p	0.012	0.011	0.01	0.009	0.009	0.009	0.009
AGT022+2.p	0.01	0.009	0.009	0.009	0.009	0.009	0.009
AGT025+2.p	0.012	0.011	0.009	0.01	0.01	0.01	0.009
SWB064+1.p	0.012	0.01	0.009	0.009	0.009	0.009	0.011
COM142+1.p	0.011	0.011	0.009	0.009	0.01	0.009	0.009
SWB043+1.p	0.011	0.01	0.01	0.009	0.009	0.009	0.009
SET720+4.p	0.01	0.01	0.009	0.009	0.009	0.009	0.009
NUN065+1.p	0.011	0.01	0.009	0.009	0.009	0.009	0.01
GEO494+1.p	0.013	0.011	0.012	0.01	0.009	0.009	0.009
SWB062+1.p	0.012	0.011	0.012	0.009	0.009	0.01	0.009
SWB016+1.p	0.011	0.01	0.011	0.01	0.009	0.011	0.01
TOP028+2.p	0.01	0.01	0.009	0.009	0.01	0.009	0.01
GEO532+1.p	0.011	0.009	0.009	0.009	0.009	0.009	0.009
COM129+1.p	0.012	0.012	0.01	0.01	0.009	0.009	0.012
LCL562+1.p	0.012	0.01	0.009	0.009	0.009	0.009	0.01
KLE147+2.p	0.012	0.011	0.01	0.009	0.009	0.009	0.009
NUM336+1.p	0.013	0.012	0.06	0.01	0.01	0.01	0.01
SWB058+1.p	0.014	0.012	0.058	0.093	0.01	0.01	0.127
SWB079+1.p	0.062	0.098	0.013	0.138	0.011	0.178	0.214

SWV053+1.p	0.013	0.012	0.065	0.138	0.1	0.173	0.211
SWB009+1.p	0.065	0.107	0.012	0.012	0.138	0.21	0.172
CSR069+5.p	0.017	0.017	0.015	0.015	0.012	0.013	0.012
LAT291+2.p	0.106	0.067	0.142	0.18	0.217	0.012	0.256
NUN056+1.p	0.058	0.13	0.167	0.094	0.205	0.244	0.012
SWB044+1.p	0.061	0.098	0.134	0.013	0.17	0.205	0.239
LCL472+1.p	0.02	0.02	0.017	0.017	0.016	0.015	0.014
REL021+2.p	0.023	0.021	0.02	0.019	0.018	0.017	0.016
NUN055+1.p	0.025	0.023	0.024	0.022	0.021	0.02	0.019
MSC010+1.p	0.029	0.028	0.023	0.028	0.028	0.02	0.029
SWB101+1.p	0.03	0.027	0.027	0.025	0.025	0.023	0.023
SYN986+1.004.p	0.03	0.029	0.028	0.026	0.025	0.025	0.024
CSR237+1.p	0.034	0.033	0.031	0.03	0.03	0.028	0.028
GEO521+1.p	0.036	0.035	0.034	0.032	0.032	0.031	0.029
CSR055+5.p	0.039	0.038	0.037	0.036	0.035	0.034	0.033
ITP013+4.p	0.043	0.042	0.041	0.04	0.039	0.037	0.037
ITP015+1.p	0.076	0.112	307.674	0.069	0.137	0.193	0.166
REL045+1.p	0.056	0.054	0.055	0.055	0.052	0.05	0.05
SCT170+2.p	0.06	0.058	0.056	0.053	0.054	0.057	0.052
NUM839+2.p	0.066	0.068	0.074	0.072	0.063	0.057	0.06
GEO507+1.p	0.009	0.009	0.01	0.009	0.008	0.009	0.008
SWB108+1.p	0.132	0.132	0.13	0.129	0.132	0.165	0.127
SWB028+2.p	0.156	0.154	0.144	0.133	0.151	0.152	0.158
ITP013+1.p	0.162	0.161	306.51	304.999	0.157	0.156	0.163
ITP023+1.p	0.2	0.196	0.17	0.152	0.198	0.189	0.193
NUM461+2.p	11.443	11.519	0.678	0.071	7.098	9.299	9.627
GRA012+1.p	0.207	0.206	0.205	0.203	0.203	0.204	0.24
GEO525+1.p	0.219	0.218	0.216	0.215	0.214	0.214	0.214
SWB056+1.p	0.24	0.239	0.238	0.237	0.235	0.236	0.233
NLP260+1.p	0.241	0.24	0.237	0.238	0.236	0.239	0.235
REL030+1.p	0.243	0.241	0.241	0.239	0.238	0.239	0.237
CSR201+1.p	0.251	0.242	0.242	0.24	0.239	0.24	0.238
SET066+1.p	0.246	0.245	0.244	0.243	0.245	0.241	0.24
CSR044+2.p	0.248	0.246	0.245	0.247	0.244	0.245	0.242
BIO005+1.p	0.267	0.265	0.264	0.262	0.262	0.261	0.262
SYO606+1.p	46.245	46.688	0.091	9.234	14.102	48.293	51.437
PRO010+3.p	0.309	0.305	0.308	0.305	0.304	0.298	0.348
ITP021+1.p	1.614	1.62	1.131	1.539	1.588	1.562	1.679
ITP012+1.p	3.415	3.575	2.631	3.586	3.472	3.525	3.846
GEO332+1.p	3.055	3.081	3.021	2.895	3.099	3.008	3.295
LCL646+1.005.p	2.906	2.948	3.112	3.04	3.104	3.109	3.15
GEO298+1.p	3.866	3.841	3.757	3.696	3.78	3.976	4.152
SWW478+7.p	5.439	5.272	5.314	5.44	5.689	5.576	6.103
SEU348+2.p	7.152	7.235	7.217	7.237	7.433	7.478	7.935
PRO003+2.p	8.063	8.129	8.135	7.832	8.217	8.032	8.659
GRP746+1.p	306.37	306.48	16.701	307.238	307.567	306.824	306.486
SWW348+1.p	18.94	19.22	19.68	18.772	19.257	19.454	20.487
SWW350+1.p	19.193	19.398	19.694	19.161	19.657	19.557	19.944
SWW337+1.p	20.907	20.797	20.613	20.449	20.679	20.118	21.34

SWW339+1.p	21.705	22.06	22.238	21.848	21.897	22.057	22.561
GEO283+1.p	22.931	23.052	25.116	25.043	26.072	23.628	24.997
GEO286+1.p	28.884	29.085	30.16	29.611	31.799	29.576	31.16
GRP655+2.p	33.715	34.033	31.437	33.891	34.223	33.109	34.283
GEO281+1.p	31.721	31.889	35.234	33.036	34.847	32.933	34.539
PUZ077+1.p	50.433	50.232	35.081	42.176	45.367	48.507	52.393
GEO168+1.p	44.031	44.452	44.123	43.539	44.053	43.94	46.782
SYO605+1.p	53.539	53.839	50.49	52.702	53.238	53.46	55.582
LCL403+2.p	57.142	58.323	58.932	57.116	58.734	57.759	59.571
SET671+3.p	113.988	112.306	304.77	83.846	111.828	111.754	116.296
LCL642+1.010.p	81.422	82.259	84.527	83.955	84.406	86.174	91.24
SWW470+1.p	227.801	224.679	89.866	238.68	226.527	222.825	238.192
GRP654+2.p	95.475	96.542	89.605	90.596	92.278	95.299	99.265
GRP664+1.p	113.49	112.5	105.474	107.302	102.901	110.015	114.988
MGT005+2.p	216.325	217.314	113.361	110.072	114.986	114.329	121.136
LCL642+1.015.p	122.743	123.628	126.232	126.597	126.383	130.866	137.397
SWW893+1.p	140.615	140.882	307.357	124.601	128.324	138.03	146.766
SWW884+1.p	151.058	151.878	143.756	141.206	142.347	149.41	155.895
LCL656+1.015.p	164.727	164.912	165.782	166.106	168.33	170.198	178.473
GEO310+1.p	222.817	224.626	216.924	235.653	242.102	229.157	239.72
SCT117+1.p	293.299	293.225	221.799	255.493	264.382	285.535	305.842
GEO311+1.p	298.235	302.223	273.717	283.145	293.984	304.098	303.921
GEO329+1.p	284.742	286.206	303.5	303.353	299.943	297.152	303.495
LCL656+1.020.p	300.287	300.269	300.266	300.276	300.244	300.276	300.248
ALG195+1.p	300.599	300.419	301.051	301.064	301.021	301.044	300.965
LAT378+1.p	300.795	300.856	341.525	350.002	300.876	300.907	300.726
LCL652+1.015.p	300.901	300.934	300.788	300.919	300.951	300.979	300.989
BOO109+1.p	301.644	301.66	301.64	301.634	301.63	301.635	301.641
GEO317+1.p	302.592	302.344	302.368	301.938	302.442	302.522	302.181
SYO604+1.p	302.372	302.454	303.198	303.182	303.536	303.378	303.267
GEO273+1.p	303.086	303.195	302.859	302.922	302.857	302.905	302.753
GEO167+1.p	303.388	303.461	304.284	304.159	305.948	304.457	304.4
GEO295+1.p	304.372	304.345	303.523	304.56	304.405	304.214	304.78
GEO323+1.p	303.563	303.552	303.542	303.539	303.846	304.139	304.185
GEO292+1.p	303.719	303.648	303.687	303.744	303.771	303.557	304.01
GRP720+1.p	304.635	304.712	303.92	304.196	304.097	304.927	304.403
LCL897+1.p	304.485	304.254	304.48	304.707	304.528	304.265	304.28
GRP666+6.p	305.34	304.432	304.383	304.755	304.561	304.571	305.369
SET144+3.p	304.756	304.789	304.828	304.866	304.638	304.927	304.845
NUM926+3.p	305.539	305.416	305.368	306.858	304.964	305.556	304.837
SEU356+2.p	308.786	305.312	305.144	305.233	305.344	305.201	304.945
NUM708+4.p	315.615	321.934	305.057	305.049	306.984	307.548	323.141
SCT115+1.p	305.618	305.584	307.359	305.75	305.455	305.563	305.089
SCT123+1.p	306.276	306.27	305.129	305.423	306.502	306.327	305.736
NUM706+4.p	306.914	307.346	305.324	306.026	305.725	323.55	307.946
ITP004+1.p	305.828	306.083	305.41	306.21	305.799	305.891	305.346
SEU232+3.p	305.471	305.49	306.278	306.7	306.329	306.842	306.515
NUM926+2.p	305.613	305.696	307.166	306.12	307.365	305.828	306.506
SCT121+1.p	306.713	306.63	306.305	307.912	306.611	306.703	306.19

SCT120+1.p	306.618	306.716	306.284	306.893	306.637	309.376	306.308
SEU237+1.p	307.287	306.528	309.577	309.16	307.242	307.554	307.005
SWW474+5.p	307.232	308.702	308.271	309.469	308.036	309.087	307.991
LCL672+1.015.p	312.12	312.367	309.855	329.171	311.537	325.739	325.028
SEU170+2.p	310.649	310.757	310.863	311.534	313.074	311.591	311.456
HWV096+1.p	350.423	350.406	350.407	350.426	350.406	350.422	350.405